

DualDeadline: Component-Staged Expert Prefetching for Exact Offloaded Mixture-of-Experts Inference

Anonymous Authors

Abstract

Expert prefetch predictors usually treat an expert as an indivisible transfer. We ask whether the structure already present inside a gated mixture-of-experts (MoE) feed-forward block creates a second, useful transfer deadline. DualDeadline predicts only the gate and up projections before the authoritative router fires. Once the native top- k route is known, it transfers the down projections while gate/up computation executes. Prediction remains advisory: misses are loaded on demand and the frozen router and expert arithmetic are unchanged.

We evaluate this post-hoc design on OLMoE-1B-7B-Instruct using autoregressive router traces and pinned-host-memory measurements from one PCIe-attached NVIDIA H200. A preregistered 120-prompt pilot reaches 45.23% held-out expert recall with four candidates, versus 26.74% for training-set popularity; a prompt-disjoint 300-prompt replication reaches 46.63%. At an equal four-expert byte budget, trace replay estimates 1.223 ms exposed stall for monolithic prefetch and 1.100 ms for component staging. A real CUDA-stream microbenchmark with eight OLMoE experts reduces block latency from 2.193 ms to 1.939 ms without numerical change. Benefits persist under prompt-cold LRU caches from 8 to 64 experts per layer. We distinguish measured microbenchmarks from trace-driven simulation and do not claim end-to-end serving speedup.

1 Introduction

Sparse MoE models increase total capacity while activating only a few experts per token [1, 2]. When the expert pool does not fit accelerator memory, inactive weights can remain in host memory and be copied on demand. The native route, however, is available only after the router executes, placing host-to-device transfer on the decode critical path.

SpecPrefetch [3] introduced a clean transfer-only design: a small post-hoc adapter predicts next-layer experts early, but the native router still controls execution. False predictions therefore waste bandwidth rather than change logits. Its scheduler reasons about whether a whole predicted expert can arrive before execution. We investigate an orthogonal question: *must the entire expert share one deadline?*

A gated expert naturally contains three matrices:

$$y_e = W_d^e [\text{SiLU}(W_g^e x) \odot W_u^e x]. \quad (1)$$

For OLMoE, W_g and W_u together occupy two thirds of the

expert bytes; W_d occupies one third. The down matrix is not consumed until after gate/up arithmetic has produced the intermediate activation. This dependency creates a second transfer-compute overlap opportunity.

Our contributions are:

- a router-preserving, component-staged schedule that speculates on gate/up weights and transfers authoritative down weights during compute;
- deadline-aware evaluation that charges predictor latency and uses measured transfer and layer windows instead of recall alone;
- a preregistered held-out study, a prompt-disjoint replication, cache sensitivity, and a real-stream overlap microbenchmark on an H200; and
- bounded forward-pass optimization experiments, including algebraic low-rank materialization, CUDA Graphs, `torch.compile`, and a custom Triton GEMV, with negative results reported.

2 Background and Related Work

Router-preserving prefetch. SpecPrefetch predicts the expert set at layer $l + 1$ from router input at layer l , then schedules only transfers that can finish within the measured window [3]. FATE instead exploits adjacent-layer gate regularity without learned parameters [4]. ExpertFlow adapts prediction horizon to runtime conditions [5], while OD-MoE predicts multiple layers ahead in a distributed cacheless setting [6]. We retain the next-layer, transfer-only contract and change expert transfer granularity.

Earlier same-layer information and finer caching. Pre-attention prediction learns same-layer routes from the state before attention [7]; we include this as an exploratory horizon control rather than a novelty claim. SliceMoE caches bit slices at different precision [8]. Our components follow the exact computational dependency in Eq. 1; no quantization or approximate execution is introduced.

3 DualDeadline

3.1 Transfer-only prediction

Let $\mathcal{T}^{l+1}$ be the native top- k expert set and $s^{l+1} \in \mathbb{R}^E$ be advisory scores predicted from layer- l router input. The executed set remains $\mathcal{T}^{l+1}$ for every policy. A missing component is synchronously copied before use, hence prediction errors cannot change the mathematical forward pass.

Our primary predictor is shared across layers. It applies layer normalization, a width-256 hidden projection, a 32-dimensional target-layer embedding projected to width 256, GELU, and a 64-way output. It has 553,536 trainable parameters; the base model is frozen. Multi-label binary cross entropy uses the native top-8 route as target. Checkpoint selection maximizes validation ready recall after measured predictor and transfer deadlines.

3.2 Two component deadlines

Before the target router, the scheduler copies predicted (W_g, W_u) pairs in score order. Let G be those ready pairs and let $\mathcal{T}$ be the native route revealed at the target router. Experts in $G \cap \mathcal{T}$ can start gate/up compute immediately. Down matrices are never selected by the speculative predictor: after routing, the scheduler copies $\{W_d^e : e \in \mathcal{T}\}$ on a DMA stream while gate/up kernels execute on a compute stream. Gate/up misses are copied on demand. With one DMA channel, our conservative replay model is

$$S_{\text{stage}} = (k - |G \cap \mathcal{T}|) t_{gu} + \max(0, k t_d - t_{\text{compute}, gu}), \quad (2)$$

before persistent-cache hits are removed. The monolithic comparison charges $(k - |F \cap \mathcal{T}|) t_{\text{full}}$ for ready full experts F . Both receive the same speculative byte budget.

3.3 Runtime and cache model

We subtract batch-1 predictor latency from the measured issue-to-router window. Transfers are feasible only if their estimated completion precedes the target router. Cache replay uses an independent full-expert LRU per layer, reset at each prompt. Speculative components use a separate bounded workspace and unused predictions are discarded at the target layer. This isolates prefetch value from cache pollution; alternative shared-pool policies remain future work.

4 Experimental Method

Model and hardware. OLMoE-1B-7B-Instruct has 7B total and about 1B active parameters [9]. It contains 16 MoE layers, 64 experts per layer, and activates 8 experts per token. Experiments use bfloat16 on one NVIDIA H200 with 141 GB HBM attached over PCIe Gen5 x16. We pin host tensors and use PyTorch 2.9.1, CUDA 12.8, Transformers 5.14.1, and Triton 3.5.1.

Data and traces. The preregistered pilot contains 80 GSM8K and 40 HumanEval prompts, split 60/20/20 by prompt. Greedy autoregressive decoding produces 10,872 decode steps; the 24-prompt test set contains 32,520 target layer-token pairs. After the pilot result, we add a clearly labeled exploratory replication with 200 GSM8K and 100 HumanEval prompts whose IDs are disjoint from the pilot. Its 60-prompt test set contains 81,195 pairs. Native routes, full router logits, router inputs, and layer-start states are

Table 1: Pilot held-out expert recall (%). “Rep.” is the disjoint replication for the primary learned predictor.

Policy	R@2	R@4	R@8
Random	3.16	6.24	12.41
Previous-layer route	3.09	6.09	12.24
Training popularity	16.34	26.74	41.15
Learned (primary)	23.69	45.23	76.33
Oracle	25.00	50.00	100.00
Learned (Rep.)	24.15	46.63	79.77

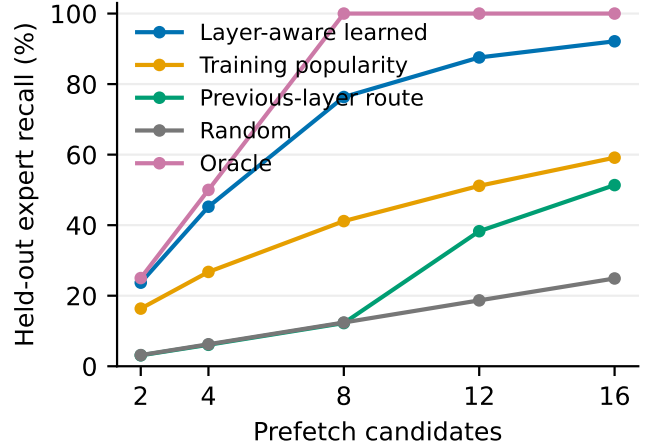


Figure 1: Held-out pilot recall under a fixed candidate budget.

detached and stored; prompts, never tokens, are the split and bootstrap unit.

Policies and statistics. We compare random scores, training-set popularity, previous-layer native routes (training-free), learned prediction, and an oracle. Reported intervals are paired 2,000-resample percentile bootstraps over prompts. The simulator uses measured p50 layer windows and a linear fit to pinned host-to-device copies. Results explicitly labeled “simulation” are not end-to-end serving measurements.

5 Results

5.1 Held-out expert prediction

Table 1 shows that hidden-state prediction captures information not present in previous expert IDs alone. At budget four, learned-minus- popularity is +18.49 percentage points in the pilot (95% CI [+17.68,+19.27]) and +20.45 points in the replication ([+19.82,+21.04]). Pilot R@4 is 44.43% on GSM8K and 45.15% on HumanEval for the earlier direct-embedding control; the primary conformant checkpoint shows the same qualitative task stability.

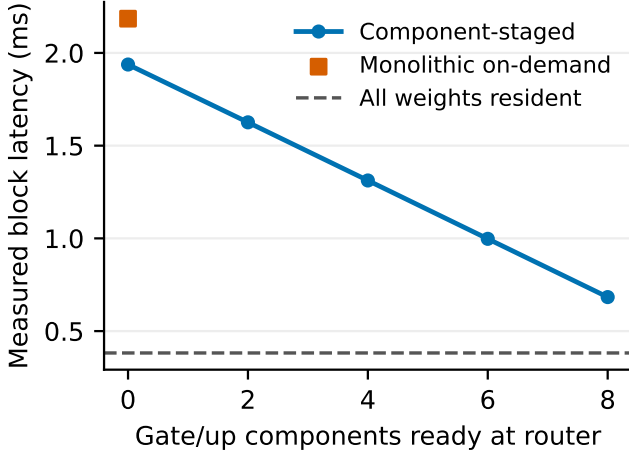


Figure 2: Real pinned-memory copies and CUDA streams using eight layer-0 OLMoE experts. Outputs match resident execution exactly.

5.2 Measured component overlap

Pinned-copy p50 times are 0.159 ms for an 8 MiB gate/up component, 0.083 ms for a 4 MiB down component, and 0.234 ms for a 12 MiB full expert. Eight experts’ gate/up and down arithmetic take 0.257 ms and 0.128 ms, respectively. The next-layer issue-to-router window is 0.81–0.89 ms before predictor cost; the primary predictor costs 0.092 ms. Thus four gate/up transfers are feasible on 14 of 15 target layers and five on one layer.

With real tensors, resident compute takes 0.382 ms, monolithic copy then compute takes 2.185 ms, and serial component reordering takes 2.194 ms. Actual two-stream staging takes 1.938 ms. Therefore concurrency, not re-ordering, saves 0.247 ms in this run. Making 2, 4, 6, and 8 gate/up components ready before the router reduces latency monotonically to 1.626, 1.312, 0.998, and 0.683 ms. All strategies are numerically identical to resident execution. Repeating the full-transfer comparison independently on all 16 layers gives a median 2.192 ms for monolithic loading and 1.938 ms for staging, a median saving of 0.254 ms (11.6%); the layerwise saving ranges from 0.247 to 0.390 ms, with zero numerical error. At layer 0, varying routed top- k from 1, 2, 4, to 8 gives absolute staging savings of 0.063, 0.061, 0.122, and 0.254 ms, respectively.

5.3 Trace-driven stall and cache sensitivity

At four full-expert byte equivalents, monolithic replay exposes 1.223 ms per layer-token and staging exposes 1.100 ms; the paired difference is -0.123 ms (95% CI [-0.125, -0.122]). The disjoint replication gives 1.207 ms and 1.081 ms, a -0.126 ms difference ([-0.1265, -0.1252]). Larger byte budgets do not help because the measured deadline, rather than capacity, is binding. Replacing the mean-copy fit with a p95-copy fit preserves a -0.112 ms staging advantage.

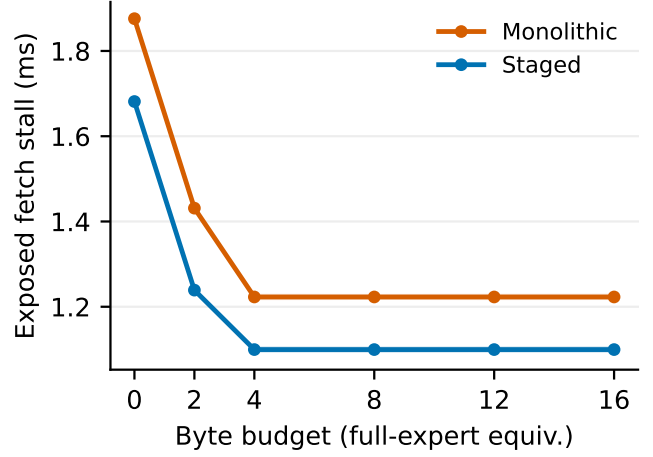


Figure 3: Cacheless pilot replay for the primary learned predictor. Bands are 95% prompt-bootstrap intervals and are often narrower than the line.

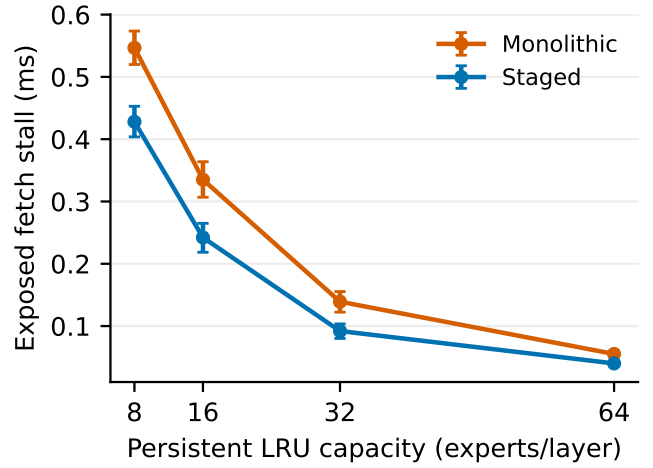


Figure 4: Prompt-cold per-layer LRU sensitivity at byte budget four.

The primary conformant checkpoint shows that staged benefit persists from 8 to 64 cached experts per layer: at capacity 8, exposed stall is 0.547 ms monolithic and 0.428 ms staged; at capacity 64 it is 0.055 ms and 0.040 ms. Intermediate capacities 16 and 32 give 0.335/0.242 ms and 0.139/0.092 ms, respectively.

5.4 Predictor and forward-pass ablations

Deadline-weighted BCE is effectively tied with and slightly below uniform BCE (44.65% versus 44.67% R@4 in the direct-embedding control), so we retain the simpler loss. Across the primary conformant checkpoint and two additional training seeds, pilot R@4 is 45.69% with a 0.40-point sample standard deviation; the corresponding disjoint-replication result is 46.92% with a 0.25-point standard deviation.

For the linear low-rank control, the two matrices can be multiplied once after training. This exact materialization

Table 2: Exploratory predictor controls on the pilot.

Predictor	Parameters	R@4 (%)
Layer-aware, width 64	140,352	39.02
Layer-aware, width 128	276,544	42.41
Layer-aware, width 256 direct emb.	548,928	44.67
Layer-aware, width 256, emb. 32	553,536	45.23
Shared low-rank, KL	540,672	34.67
Shared low-rank, BCE	540,672	35.53

reduces batch-1 p50 from 0.0388 ms to 0.0212 ms. A tuned custom Triton GEMV reaches 0.0273 ms and is faster than the factorized adapter but slower than PyTorch’s materialized GEMV; maximum error is below 3×10^{-7} . On the primary conformant layer-aware model, `torch.compile` is slower in an isolated run (0.140 ms versus 0.092 ms eager). In contrast, a static CUDA Graph costs 0.0220 ms to replay and 0.0325 ms including device-side input copies, versus 0.0938 ms eager, with identical output. The primary deadline analysis conservatively retains the measured eager cost. These results argue for measuring small-kernel launch behavior rather than assuming a compiler or custom kernel must win [10].

6 Limitations

Our end-to-end claims are deliberately narrow. Trace replay assumes one DMA channel and uses measured p50 windows; only the expert-block overlap is executed with real streams. We do not yet integrate staged components into a production scheduler, model PCIe contention from other requests, or measure batched serving throughput. The cache simulator gives speculative weights a separate workspace and resets LRU state per prompt. OLMoE is smaller than current frontier MoEs, and results must not be extrapolated to trillion- parameter models without measurement. The disjoint replication, low-rank control, and several systems ablations were added after the preregistered pilot and are labeled exploratory in the machine-readable protocol.

7 Conclusion

An expert need not have one transfer deadline. Moving gate/up weights first and authoritative down weights during gate/up computation preserves exact routing while reducing speculative bytes and exposed stall. On OLMoE and an H200, the effect survives held-out prompts, a larger disjoint replication, cache sensitivity, and direct CUDA-stream measurement. The study also shows why ready recall, predictor latency, and real overlap must be reported together. A production offloading integration and validation on larger MoEs are the necessary next steps.

References

[1] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean.

Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[2] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[3] Jinwei Kong, Runqi Meng, Fanyi Wang, Wentao Qiu, Haotian Hu, Yongjian Zhou, and Zhenhua Ge. SpecPrefetch: Parameter-efficient expert prefetching for sparse MoE foundation models. *arXiv preprint arXiv:2607.24787*, 2026.

[4] Zhiyuan Fang, Zicong Hong, Yuegui Huang, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. Accurate expert predictions in MoE inference via cross-layer gate. *arXiv preprint arXiv:2502.12224*, 2025.

[5] Zixu Shen, Kexin Chu, Yifan Zhang, Dawei Xiang, Runxin Wu, and Wei Zhang. ExpertFlow: Adaptive expert scheduling and memory coordination for efficient MoE inference. *arXiv preprint arXiv:2510.26730*, 2025.

[6] Liujuanfu Wang, Yuyang Du, Yuchen Pan, Soung Chang Liew, Jiacheng Liu, and Kexin Chen. OD-MoE: On-demand expert loading for cacheless edge-distributed MoE inference. *arXiv preprint arXiv:2512.03927*, 2025.

[7] Shien Zhu, Samuel Bohl, Robin Oester, and Gustavo Alonso. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv preprint arXiv:2511.10676*, 2025.

[8] Yuseon Choi, Sangjin Kim, Jungjun Oh, Gwangtae Park, Byeongcheol Kim, and Hoi-Jun Yoo. Slice-MoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference. *arXiv preprint arXiv:2512.12990*, 2025.

[9] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, et al. OLMoE: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.

[10] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.